DocuMind AI -- Benchmark Report

Generated: 2026-10-12T19:35:16.590935+00:00 | Python 3.12.3 | Embedding provider: local-hashed-bow | LLM judge: heuristic (no API key set)

Corpus: 60 synthetic documents, 542 chunks, 180 golden questions.

All figures in this report are computed directly from the evaluation harness run referenced above and are reproducible by re-running run_eval.py. Numbers marked 'estimated' used a heuristic fallback (no LLM API key was set for this run) and are explicitly distinguished from 'measured' numbers throughout. This is a personal portfolio project benchmark, not a report of production system performance.

Retrieval Quality (document-scoped, hybrid dense+BM25)

Metric	Value
Recall@5	92.2%
Precision@5	18.4%
MRR	0.4502
NDCG@5	0.5722
Hit Rate	92.2%
Corpus-wide Recall@5 (harder task, for comparison)	2.2%

RAG Quality

Metric	Value
Faithfulness	0.8369 (estimated (heuristic judge, no LLM API key set))
Context Precision	19.0%
Context Recall	95.0%
Answer Relevancy	0.6910
Citation Accuracy (deterministic)	91.0%
Hallucination Detector Precision	100.0%
Hallucination Detector Recall	77.8%

Latency (retrieval-only)

Metric	Value
p50	0.116 ms
p90	0.164 ms
p95	0.184 ms
p99	0.208 ms
Throughput	8007.4 req/s

Token Cost

Metric	Value
Avg input tokens/query	295.3
Avg output tokens/query	41.6
Baseline cost/query (frontier-only)	$0.001510
Tiered cost/query	$0.000403
Cost savings from tiering	73.3%
Projected monthly cost @ 50,000 queries	$20.14

Cache Performance

Metric	Value
Backend	in-memory (no REDIS_URL set)
Hit ratio	71.2%
Simulated repeat-query rate	40%
Embedding calls saved	71.2%

Scope Notes & Honesty Disclosures

Retrieval: document

Latency: retrieval-only latency (dense+BM25 hybrid search against in-memory index); does NOT include LLM generation time or network latency -- run this against a live deployed API endpoint (see load_test.py) for full end-to-end request latency.

Ingestion: measures local embedding-generation throughput only (text already extracted). Does NOT include: PDF text-extraction time (PyMuPDF), network latency to an embeddings API if using a hosted model, or OCR time for scanned documents. OCR speed/accuracy requires real scanned samples and is intentionally not fabricated here -- see README.

Vector Search: in-memory numpy brute-force cosine search (no ANN index, no disk I/O, no network). This is a lower bound for a real Qdrant/FAISS HNSW deployment under network+disk conditions, and an upper bound for raw compute cost at this corpus size -- report both boundaries rather than implying this equals production vector-DB latency.

Cache: repeat_rate_assumption models a reviewer session pattern (re-querying an open document); it is an explicit simulation parameter, not an observed production traffic statistic -- re-run with your own real query logs once available.